

React JS

1. React , ReactDOM CDN:

- React: React is a JavaScript library used to build user interfaces (UI), especially web pages.
Example: Buttons, forms, menus, product cards, etc.
- ReactDOM: ReactDOM connects React with the HTML page (DOM).
- CDN : CDN (Content Delivery Network) lets you use React directly from the internet without installing it using npm.

Example:

```
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
```

Code Snippet:

```
var parent=document.querySelector("#parent") // it is the selector used in the
html
var root=ReactDOM.createRoot(parent)

var h1=React.createElement('h1',null,"Hello Express") // here null is nothing
but property like id , class we can add if needed
root.render(h1)
```

2. Install , import and export

- To install : npm create vite
- To export : let function be as :

```
function App(){
  return "Hello"
}
export default App;
```
- To import : import App from './App.jsx'

3. See it:

suppose there is an function changeValue if
<button onClick={changeValue()}>Change Value</button> (it automatically gets called no need to click)
<button onClick={changeValue}>Change Value</button> only when click

4. useState:

```
const [a,setA]=useState(10)
```

here a is the variable
and setA is the function used to change the variable a
and (10) is the initial value of the variable a

The setA is the async process so only :

```
const increaseCount=()=>{
  console.log("Before value: ",a);
  seta(a+1)
  console.log("After value: ",a);
}
```

both Before and After value is same but once it changed it will render the UI

code snippet:

```
import { useState } from "react"

const App = () => {
  const [a, seta] = useState(10)

  const increaseCount = () => {
    console.log("Before value: ", a);
    seta(a+1)
    console.log("After value: ", a);
  }

  return (
    <div>
      <h1>Hello value of a:{a} </h1>
      <button onClick={increaseCount}>Click to increase</button>
    </div>
  )
}

export default App
```

5. prevent Default behaviour as default when clicked submit it refreshes the page

```
const submitHandler = (e) => {
  e.preventDefault()
  console.log("hi", e)
}

<form onSubmit={(e) => { submitHandler(e)}}>
</form>
```

6. Two way binding:

setting the value using the react and changing the value using the react rather than directly interacting :

code snippet:

```
const [username, setName] = useState("")
return (
  <div>
    <form >
      <input type="text" placeholder='HELlo'
        onChange={(e) => {
          console.log(e.target.value)
          setName(e.target.value)
        }}
        value={username}></input>
      <button>Submit</button>
    </form>
  </div>
)
```

here e.target.value each time it gets the types value so here it prints in the console and also using setName it sets the same thing to the username value so two way binding is done

Topics: Components, PropDrilling

7. Axios (use to link frontend and backend)

- install using : npm i axios
- and axios returns promise so its an async so need to use await and function must be async because it need to wait until promise is done

code snippet:

```

import React, { useState } from 'react'
import axios from 'axios'

const ApiAxios = () => {

  const [data, setData]=useState([])

  const getData= async ()=>{
    // console.log("Hello")
    const response= await axios.get('https://picsum.photos/v2/list');
    console.log(response.data)
    setData(response.data)
  }
  return (
    <div>
      API calling using Axios
      <br></br>
      <button onClick={getData}>GET DATA</button>
      <div>
        {data.map(function (ele, idx) {
          return (
            <div key={idx}>
              <img
                style={{ height: "100px", width: "100px" }}
                src={ele.download_url}
                alt="nope"
              />
              <h1>{ele.author}</h1>
            </div>
          );
        })}
      </div>
    </div>
  )
}

export default ApiAxios

```

8. React Router DOM

- by default in the react there is no faculty of routing of one page to another
- install : npm i react-router-dom
- It consists of Three Steps

Step 1: First wrap up the main.jsx in the BrowserRouter

code snippet:

```

import {BrowserRouter} from 'react-router-dom'

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <ReactRouterDOM />
  </BrowserRouter>
)

```

Step 2: in the App.jsx use Routes and Router

in each Route it consists of path and element where path is the url hitting and the element is the which one should be called

code snippet:

```

<Routes>
  <Route path='/about' element={<About />} />
</Routes>

```

Step 3: Link to the Route

- Route is only the destination where as link is the path to destination

code snippet:

```
<Link to="/about">About</Link>
```

Full code:

```
import React from 'react'
import { Route, Routes, Link } from 'react-router-dom'
import About from './pages/About'
import Home from './pages/Home'
import Details from './pages/Details'
import Contact from './pages/Contact'
import Product from './pages/Product'

const ReactRouterDOM = () => {
  return (
    <div>

      <Link to="/about">About</Link>
      <Link to="/contact">Contact</Link>
      <Link to="/product">Product</Link>
      <Link to="/details"> Details</Link>

      <Routes>
        <Route path="/about" element={ <About /> } />
        <Route path="/" element={ <Home /> } />
        <Route path="/details" element={ <Details /> } />
        <Route path="/contact" element={ <Contact /> } />
        <Route path="/product" element={ <Product /> } />
      </Routes>
    </div>
  )
}

export default ReactRouterDOM
```

9. ContextApi (state management tool)

- Context API allows us to share data across multiple components without manually passing props through every intermediate component (avoids prop drilling).
- It is commonly explained in 3 steps.

Step 1: create one create (like useContext, storageContext, employeeContext)

- Create a context instance using createContext().
- Create a Provider component that provides the data.
- children is a prop that represents the components placed inside the Provider.
- The data we want to share is passed using the value prop.
- children refers to all components/elements placed inside the <UserContext>...</UserContext>.

userContext.jsx:

```
import React, { createContext } from 'react'
```

```
export const dataContext= createContext()
```

```
const UserContext = ({children}) => {
```

```
  const user={
    username:"something",
    age: 10,
    city: "None"
```

```

    }
    return (
      <div>
        <dataContext.Provider value={user}>
          {children}
        </dataContext.Provider>

        In useContext.jsx
      </div>
    )
  }
}

export default useContext

```

Step 2:

- wrap by the main.jsx with the context that is created and also the classes which need to be used by it here Context.jsx is one of it and it act as the child to parent useContext.jsx

code snippet:

```

ReactDOM.createRoot(document.getElementById("root")).render(
  <UserContext>
    <Context />
  </UserContext>
)

```

UserContext

```

  â†“
  Context
  â†“
  Header

```

Step 3:

- usage in the child one as

```

import React, { useContext } from 'react'
import { dataContext } from '../context/UserContext'

const Header = () => {

  const data = useContext(dataContext)

  return (
    <div>
      Header from useContext {data.age}
    </div>
  )
}

export default Header

```

Hooks:

already useState and the useContext is completed rest of the hooks are listed down:

1. useState:

using object:

```

import React from 'react'
import { useState } from 'react'
// useState with number and strings
const useState = () => {
  const [details, setDetails]=useState({counter:0,name:""});
}

```

```

const increaseCount=()=>{
  setDetails((prev)=>({
    ...prev,
    counter: prev.counter+1
  }));
}

const decreaseCount=()=>{
  setDetails((prev)=>({
    ...prev,
    counter: prev.counter+1
  }));
}

const setText=(e)=>{
  setDetails((prev)=>({
    ...prev,
    name:e.target.value
  }));
}

return (
  <div>
    <input type="text" onChange={setText}></input>
    <h1>{details.name} Count value is : {details.counter}</h1>
    <button onClick={increaseCount}>Click to increase</button>
    <button onClick={decreaseCount}>Click to decrease</button>
  </div>
)
}

```

export default UseState

2. useEffect:

- useEffect(callback , dependencies) // here callback is what to run and dependencies is when to run
- 3 types are there:
 - a. useEffect without dependencies
 - b. useEffect with empty array
 - c. useEffect with variable

```
import React from 'react'
```

```
import { useState,useEffect } from 'react'
```

```

const UseEffectOne = () => {
  const [data,setData]=useState(0);
  const [otherdata,setOtherData]=useState(5);

```

// without dependency it running when every the change happens in this component (i.e UseEffectOne)

```

  useEffect(()=>{
    document.title=`${data} is clicked!`
  })

```

```

  // with empty array only for one time
  useEffect(()=>{
    document.title=`${data} is clicked!`
  },[])

```

```

  // with variable
  useEffect(()=>{
    document.title=`${otherdata} is clicked!`
  },[otherdata])

```

```

    const increaseCount={()=>{
      setData(data+1);
    }}

    const decreaseCount={()=>{
      setData(data-1);
    }}

    return (
      <div>

        <h1> Count value is : {data}</h1>
        <button onClick={increaseCount}>Click to increase</button>
        <button onClick={decreaseCount}>Click to decrease</button>

        <br></br>
        <h1>Other Count value is : {otherdata}</h1>
        <button onClick={()=>{setOtherData(otherdata+5)}}>Click to increase by
5</button>
      </div>
    )
  }
}

```

export default UseEffectOne

3. useContext: refer to the top one only

4. useRef:

- used to access the dom elements
- create mutable variable without re-rendering

```
const variableName=useRef(optinimalValue);
```

codesnippet:

a.used to access the dom elements

```

import React from 'react'
import { useEffect } from 'react'
import { useRef } from 'react'
import { useState } from 'react'

const UseRef = () => {

  const inputEle=useRef();

  // React stores the input DOM element inside current
  const handleClick={()=>{
    console.log(inputEle.current)
    inputEle.current.style.width="500px"
    inputEle.current.focus()
  }}

  return (

    <div>
      <input type="text" ref={inputEle}></input>
      <br>
      </br>
      <button onClick={handleClick}>Click here </button>

    </div>
  )
}

```

```
export default UseRef
```

b. create mutable variable without re-rendering

```
import React from 'react'
import { useEffect } from 'react'
import { useRef } from 'react'
import { useState } from 'react'

const UseRef = () => {
  const [ name, setName]=useState("")

  // useRef returns the object with consists of current
  const count=useRef(0);

  useEffect(()=>{
    count.current=count.current+1;
  })

  return (
    <div>
      <input type="text" onChange={(e)=> setName(e.target.value)}></input>
      <h1> Name is: {name}</h1>
      <h1> Renders: {count.current}</h1>
    </div>
  )
}
```

```
export default UseRef
```

5. UseReducer

- use to reduce complex state function
- use to reduce more number of state function

useReducer take two input arguments

1. reducer function: the logic of what to do
2. initialState: initial value

UseReducer returns array of two elements:

1. state: current state of that thing
2. dispatch: update that state

Reducer function have two parameter:

1. state: current state that is been used
2. action: what need to be done for the current stage (what ever is passed in the dispatch function can be accessed in the action to perform various action in the reducer accordingly)

code snippet:

```
import React from 'react'
import { useReducer } from 'react';
```

```
const ACTION={
  INCREASE:"increase",
  DECREASE:"decrease"
}
```

```
const initialState={count:0};
```

```
const reducer =(state,action)=>{
  switch(action.type)
```

```

    {
      case ACTION.INCREASE:
        return {count:state.count+1};
      case ACTION.DECREASE:
        return {count:state.count-1};
      default:
        return state
    }
  }
}

const UseReducer = () => {

  const [ state,dispatch]=useReducer(reducer,initialState);

  const increaseCount=()=>{
    dispatch({type:ACTION.INCREASE})
  }

  const decreaseCount=()=>{
    dispatch({type:ACTION.DECREASE})
  }
  return (
    <div>
      <h2>Count: {state.count}</h2>
      <button onClick={increaseCount}>Increase</button>
      <button onClick={decreaseCount}>Decrease</button>
    </div>
  )
}

```

export default UseReducer

6. UseLayoutEffect:

- it is similar to useEffect but runs before the dom printing
- used to access the layout info like style height width details
- it is sync only execute next line when current line is done

import React, { useEffect, useLayoutEffect, useState,useRef } from 'react'

```

const UseLayout = () => {
  const [toggle, setToggle] = useState(false)
  const dom=useRef()

  useLayoutEffect(() => {
    if(dom.current!=null)
    {
      const dimension=dom.current.getBoundingClientRect();
      dom.current.style.paddingTop=`${dimension.height}px`
    }
  }, [toggle])

  useEffect(() => {
    if(dom.current!=null)
    {
      const dimension=dom.current.getBoundingClientRect();
      dom.current.style.paddingTop=`${dimension.height}px`
    }
  }, [toggle])

  return (
    <div>
      <button onClick={() => setToggle(!toggle)}>Toggle</button>
      {toggle && <h4 ref={dom}>Hello</h4>}
    </div>
  )
}

```

```

    </div>
  )
}

```

export default UseLayout

Definition:

1. useState

Used to store and manage data that can change inside a component.
When the state changes, the component re-renders.

2. useEffect

Used to perform side effects like API calls, timers, or subscriptions.
It runs after the component renders.

3. useContext

Used to share data between components without passing props manually.
Commonly used for theme, authentication, and user data.

4. useRef

Used to store a value that persists between renders without causing re-rendering.
It can also be used to access DOM elements directly.

5. useReducer

Used to manage complex state using a reducer function and actions.
It is an alternative to useState for complex state logic.

5. useLayoutEffect

Runs after the DOM is updated but before the browser displays it.
Used when we need to measure or modify the DOM before the user sees it.

Other topics in react

1. JSX

- * JSX allows us to write HTML-like code inside JavaScript.
- * It makes writing UI easier and readable.
- * JSX is converted into JavaScript by React.

Code snippet:

```

```jsx
const element = <h1>Hello World</h1>
```

```

2. Props

- * Props are used to pass data from a parent component to a child component.
- * Props are read-only and should not be directly changed by the child.

Code snippet:

```

```jsx
function Child(props) {
 return <h1>Hello {props.name}</h1>
}

function App() {
 return <Child name="Sujith" />
}
```

```

3. State vs Props

* State is data managed inside a component and can be changed using a state function.

* Props are data passed from a parent component to a child component.

Simple difference:

```
```text
State â†’ managed by the component
Props â†’ received from parent
```
```

4. Conditional Rendering

* Conditional rendering means displaying different UI based on a condition.

* We can use `if`, ternary operator, or `&&`.

Code snippet:

```
```jsx
{isLoggedIn ? <h1>Welcome</h1> : <h1>Please Login</h1>}
```
```

5. Lists and Keys

* `map()` is used to display multiple elements from an array.

* `key` helps React identify which list item has changed.

Code snippet:

```
```jsx
const users = ["A", "B", "C"]

{users.map((user, index) => (
 <h1 key={index}>{user}</h1>
))}
```
```

6. Components

* Components are reusable parts of the UI.

* A component can contain HTML-like JSX, JavaScript logic, state and events.

Example:

```
```jsx
function Header() {
 return <h1>My Website</h1>
}
```
```

7. Forms

* React forms are commonly handled using state.

* The input value is stored in state and updated using `onChange`.

Code snippet:

```
```jsx
const [name, setName] = useState("")

<input
 value={name}
 onChange={(e) => setName(e.target.value)}
/>
```
```

```
/>  
...
```

This is also related to the **two-way binding** already present in your PDF.

8. Component Lifecycle

A component generally goes through three stages:

1. Mounting â†’ component is created and displayed.
2. Updating â†’ component changes because of state or props.
3. Unmounting â†’ component is removed.

`useEffect` can be used to perform actions related to these stages.

9. useMemo

- * `useMemo` is used to remember a calculated value.
- * It can avoid unnecessary calculations when a component re-renders.

Basic syntax:

```
```jsx  
const result = useMemo(() => {
 return expensiveCalculation(data)
}, [data])
```
```

10. useCallback

- * `useCallback` is used to remember a function between renders.
- * It can help avoid unnecessary function creation during re-rendering.

Basic syntax:

```
```jsx  
const handleClick = useCallback(() => {
 console.log("Clicked")
}, [])
```
```

11. Virtual DOM

- * Virtual DOM is a lightweight representation of the actual DOM.
- * When state changes, React compares the changes and updates the required parts of the actual DOM.

12. React Re-rendering

- * When state or props change, the component can re-render.
- * Re-rendering means React runs the component again and updates the UI where required.